

Relatório de decisões de implementação

Item 7 da Etapa 2 — triggers e procedures, e a escolha da ORM

Trigger ou procedure

As duas guardam lógica no banco, e a diferença está em quem decide executá-las. Procedure é chamada: existe um ponto no código da aplicação que resolve invocar. Trigger é disparada pelo próprio banco, em resposta a um evento, e não há como esquecer de acioná-la nem como contorná-la.

Separamos assim: regra que precisa valer sempre, independentemente de quem escreve, virou trigger; operação que alguém pede, com parâmetros, virou procedure.

Regra	Forma	Por quê
Um residente não cobre duas unidades no mesmo turno	trigger	é invariante do modelo; vale para o psql também
Todo atendimento alterado deixa rastro	trigger	auditoria que dependa de lembrança não é auditoria
Média de tempo do procedimento fica atualizada	trigger	valor derivado, tem de acompanhar a origem
Registrar atendimento com vários procedimentos	procedure	operação pedida, com parâmetros e resultado
Mover plantões de dia e turno	procedure	operação pedida, e a decisão de mover é externa
Tempo médio de espera por unidade	function	devolve conjunto de linhas, entra num SELECT

O custo da trigger é a opacidade. Quem lê o código da aplicação não vê que um INSERT em procedimento_realizado provoca um UPDATE em procedimento. Isso atrapalha na hora de depurar, e é a razão de os arquivos SQL da Etapa 2 terem comentário explicando cada efeito, e de a interface avisar, na aba Auditoria, que nenhuma rota escreve naquela tabela.

Um caso mostrou a tensão entre seguir a especificação e manter o dado correto. O enunciado pede trg_atualiza_media_procedimentos AFTER INSERT. Só com isso, corrigir o tempo de um procedimento realizado deixaria a média velha, e a API da Etapa 1 permite essa correção. Mantivemos o trigger com o nome e o escopo pedidos, e criamos um segundo, com sufixo diferente, para UPDATE e DELETE. Assim o que foi especificado está lá, e o que foi decisão nossa está identificado.

Escolha da ORM

Ficamos com o SQLAlchemy. Ele já estava no projeto desde a Etapa 1 como gerenciador de conexão, então a migração reaproveitou o engine e o pool em vez de abrir um segundo contra o mesmo banco. O enunciado o recomenda para Python. A camada Core também continua acessível junto da camada ORM, o que importou mais do que esperávamos: a mesma sessão que manipula objetos mapeados executa CALL nas procedures e SELECT nas views, sem uma segunda forma de falar com o banco.

A alternativa considerada foi o Django ORM, descartada porque traria o framework inteiro para um projeto que usa FastAPI, e porque o mapeamento dele presume que as tabelas são criadas pelas

migrações, o que conflita com o requisito de o esquema vir de scripts SQL escritos à mão.

O que a ORM melhorou

- A herança física deixou de aparecer no código. Cadastrar um residente eram três INSERT em sequência repetindo o id; passou a ser um grafo de objetos e um db.add.
- O tratamento de erro ficou em um lugar. Em vez de try/except/rollback em cada rota, um auxiliar traduz o SQLSTATE em código HTTP, e as respostas deixaram de trazer mensagem crua do driver.
- Duas operações que faltavam apareceram quase sem custo. Editar e excluir preceptor exigia, em SQL puro, mais dois blocos mexendo em três tabelas.
- Um erro sutil foi corrigido. A exclusão de paciente na Etapa 1 apagava a linha de pessoa junto, o que levaria embora o vínculo profissional de alguém que fosse as duas coisas. A versão com ORM só apaga a pessoa quando nenhum papel resta.

O que a ORM custou

O carregamento sob demanda é a armadilha principal. Percorrer uma lista lendo um campo de relacionamento gera uma consulta por item, e o código que faz isso parece inocente. Foi por isso que as listagens declaram `contains_eager` ou `selectinload` de forma explícita, e que existe um endpoint medindo a diferença: o problema é invisível até alguém contar as consultas.

Há também perda de controle sobre o SQL. A consulta do último atendimento por paciente, com quatro relacionamentos carregados adiante, gera um SELECT com mais de quarenta colunas. Em SQL escrito à mão só as colunas necessárias apareceriam. Para o volume deste projeto isso não pesa, e a legibilidade do código compensa.

Nem tudo migrou. As views e as procedures continuam sendo objetos do banco, chamados como tal. Mapear uma view como entidade não traria ganho, porque ela é somente leitura e não tem chave primária, que é o que o mapa de identidade usa para acompanhar objetos.

Concorrência

A verificação de conflito de escala saiu da aplicação. A Etapa 1 fazia um SELECT antes do INSERT, e entre os dois existe uma janela em que duas requisições simultâneas passam as duas pela checagem. Agora a garantia é a restrição do banco mais o trigger, e a aplicação apenas traduz o erro.

Sobre o controle otimista, a escolha veio da frequência esperada do conflito. Escala de plantão é editada por poucas pessoas, e disputa real é rara, situação em que o controle por versão custa nada no caso comum e só cobra quando o conflito acontece. Bloqueio pessimista faria toda edição esperar pela anterior para resolver um problema que quase nunca ocorre.

Sobre o esquema

As alterações da Etapa 2 ficaram em arquivo separado, e o `01_schema.sql` não foi tocado. Isso mantém a entrega da Etapa 1 reproduzível como foi avaliada e deixa claro no histórico onde uma etapa termina.

A coluna `atendimento.id_unidade` aceita nulo por causa dessa convivência. Os endpoints em SQL puro inserem atendimento sem informar unidade, e declarar a coluna obrigatória os quebraria. O efeito é visível na interface: com a chave em Etapa 1, um atendimento criado pela tela não entra nas

estatísticas mensais. Se as rotas da Etapa 1 fossem retiradas, a coluna passaria a NOT NULL.

Limitações conhecidas

- A média de tempo do procedimento é recalculada com um AVG completo a cada escrita. Com volume maior, o caminho seria manter soma e contagem incrementais.
- A exclusividade entre preceptor e residente, que o DER descreve, continua sem garantia no banco: nada impede inserir o mesmo profissional nas duas tabelas. Fechar isso exigiria um trigger ou uma coluna de papel, e não foi pedido em nenhuma das etapas.
- As credenciais do banco estão escritas em database.py. Para uso fora da disciplina, deveriam vir de variável de ambiente.
- A simulação de concorrência grava e apaga dados reais. Ela é segura para o banco de teste, mas não deveria ser exposta num ambiente com dados de verdade.